

Programación Avanzada

Técnicas de Solución de problemas.

Divide y vencerás.

Dividir un problema en problemas más pequeños y simples hasta encontrar una solución trivial o conocida.

La solución al problema general es la unión o combinación de las soluciones parciales de cada uno de los problemas pequeños.

Características que debe tener un problema P con solución S para que se pueda aplicar la técnica de Divide y vencerás:

-  El problema P se puede dividir en p_i problemas.
-  Cada uno de los p_i problemas, son similares y más simples (de complejidad menor).

Programación Avanzada

- ✚ Cada *pi* problema se puede resolver con esta misma técnica de manera recursiva.
- ✚ De esta forma, cada *pi* problema tiene una solución *si*.
- ✚ La solución *S* al problema inicial *P*, se puede obtener a partir de la *unión o combinación* de cada solución *si*.

Consideraciones.

- ✚ Esta técnica asume que existe una solución y encuentra sólo una.
- ✚ La operación de “*unión o combinación* de cada solución *si*” depende de la naturaleza de cada problema en particular y debe ser eficiente.
- ✚ Ver diagrama del algoritmo en:

Algoritmo BackTracking y Divide-Venceras.pdf

Programación Avanzada

Ejemplo de problemas que aplican la técnica de Divide y vencerás.

- ✚ Algoritmo de ordenación QuickSort.
- ✚ Búsqueda Binaria en una lista ordenada.
- ✚ Multiplicación de dos números muy grandes.

En algunos casos, la solución *si* de cada problema *pi* se puede resolver en forma concurrente.

- ✚ Algoritmo de ordenación QuickSort.

```
ordenaQS( x:xs ) = ordenaQS (menores) ++ [x] ++ ordenaQS (mayores)
  where menores = [ e | e <- xs, e < x ]
        mayores = [ e | e <- xs, e >= x ]
ordenaQS [] = []
```

Programación Avanzada

- + Ver implementación en *Qsort.pas*.
- + Ver código *Haskell* y comparar los paradigmas de programación.
- + Algoritmo de búsqueda Binaria en una lista ordenada.

Binario(Izq, Der, X)

Comienza

Si Izq < Der entonces

Med = (Izq + Der) / 2

Si X > Elem[Med] entonces

Binario(Med, Der, X)

Si no entonces

Binario(Izq, Med, X)

Si no entonces si X = Elem[Izq]

Imprime "Encontrado en la posición" Izq

Si no entonces

Imprime "El elemento buscado no está"

Termina

Programación Avanzada

- ✚ Algoritmo de búsqueda Binaria en una lista ordenada, implementación.

```
function BusquedaBinaria (inicio : integer; fin : integer; dato : integer; var DAT : array of integer ) :  
integer;  
    var  
        pivote : integer;  
    begin  
        if (inicio < fin) then  
            begin  
                pivote := (inicio + fin) div 2;  
                if dato > DAT[pivote] then  
                    result := busquedaBinaria( pivote + 1, fin, dato, DAT );  
                else  
                    if dato < DAT[pivote] then  
                        result := busquedaBinaria( inicio, pivote-1, dato, DAT )  
                    else  
                        if dato = DAT[pivote] then  
                            result := pivote  
                        end;  
                    else  
                        result := -1;  
                    end;  
                end;  
            end;  
        end;  
    end;
```

Programación Avanzada

✚ Multiplicación de dos números muy grandes.

Problema:

Encontrar un algoritmo para multiplicar dos números enteros muy grandes.

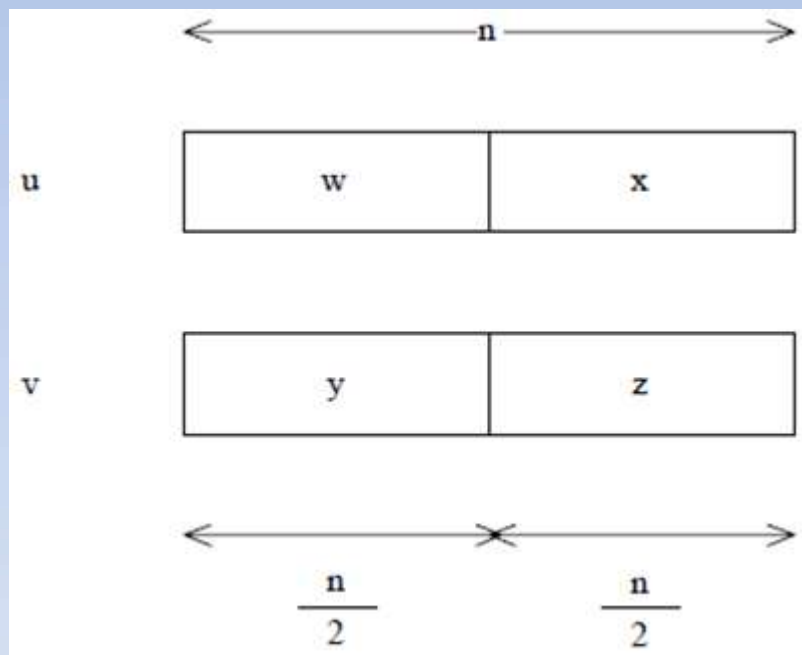
Por ejemplo:

$$3272362332323 \times 8876867324324 = ?$$

Usando la técnica de divide y vencerás.

Programación Avanzada

Solución: considérense dos números enteros, u y v , de n dígitos cada uno.



Los números u y v se pueden expresar como la suma de dos números que puedan multiplicarse más fácilmente.

Programación Avanzada

Algoritmo: Descomponer los dos números u y v en la suma de otros dos. Sea S la parte entera de $n/2$, la descomposición consiste en dividir los números entre 10^S :

$u \rightarrow w$ representa el cociente y x el resto $\rightarrow u = 10^S \cdot w + x$

$v \rightarrow y$ representa el cociente y z el resto $\rightarrow v = 10^S \cdot y + z$

entonces:

$$u \cdot v = (10^S \cdot w + x) \cdot (10^S \cdot y + z) = 10^{2S} \cdot w \cdot y + 10^S \cdot (w \cdot z + x \cdot y) + x \cdot z$$

Enseguida se calculan las suboperaciones $w \cdot y$, $w \cdot z$, $x \cdot y$, $x \cdot z$.

Si los operandos w , x , y , z son:

Programación Avanzada

- + “pequeños”, entonces se multiplican de la forma clásica.
- + “muy grandes”, se aplica de nuevo el algoritmo y se calculan recursivamente las multiplicaciones de los respectivos sumandos.

Programación Avanzada

```
Fun Mult ( u, v : entero_grande ) regresa entero_grande;
```

```
var
```

```
  n, s : ent;
```

```
  w, x, y, z : entero_grande;
```

```
  n := Max( tamaño(u), tamaño(v) );
```

```
  Si n es pequeño entonces
```

```
    regresa MultClásica( u, v ) ;
```

```
  Sino
```

```
    s := n div 2;
```

```
    w := u div  $10^S$ ;
```

```
    x := u mod  $10^S$ ;
```

```
    y := v div  $10^S$ ;
```

```
    z := v mod  $10^S$ ;
```

```
  FinSi
```

```
    regresa Mult( w, y ) ·  $10^{2S}$  +  $10^S$  · (Mult( w, z ) + Mult( x, y )) + Mult( x, z );
```

```
  FinFun.
```

Programación Avanzada

Por ejemplo:

$$\begin{aligned} 56781 \times 81765 &= (567 \times 10^2 + 81) \times (817 \times 10^2 + 65) \\ &= 10^4 \cdot 567 \times 817 + 10^2 \cdot (567 \times 65 + 817 \times 81) + 81 \times 65 \\ &= 10^4 \cdot (56 \times 10 + 7) \times (81 \times 10 + 7) + \dots \\ &= 10^4 \cdot (10^2 \cdot 56 \times 81 + 10 \cdot (56 \times 7 + 7 \times 81) + 7 \times 7) + \dots \\ &= 10^4 \cdot (10^2 \cdot (5 \times 10 + 6) \times (8 \times 10 + 1)) + \dots \\ &= 10^4 \cdot (10^2 \cdot 10^2 \cdot 5 \times 8 + 10 \cdot (5 \times 1 + 6 \times 8) + 6 \cdot 1 + \dots) \\ &= 10^4 \cdot (10^2 \cdot 10^2 \cdot 40 + 10 \cdot (5 + 48) + 6 + \dots) \end{aligned}$$

Programación Avanzada

Técnicas de Solución de Problemas.

Backtracking (Vuelta atrás, *depth-first*).

- ✚ Método sistemático que itera través de todas las combinaciones posibles de búsqueda de solución.
- ✚ Puede encontrar todas las soluciones existentes , la primera o la mejor, o bien puede establecer que no hay solución, de manera sistemática y organizada. Realiza una búsqueda completa y exhaustiva.
- ✚ Es un algoritmo que se basa en prueba y error, por lo que en general puede no ser muy eficiente. Puede tomar mucho tiempo encontrar cualquiera de ellas.

Programación Avanzada

- ✚ Para llegar a una solución, la mejor o todas, es necesario pasar por una serie de decisiones adecuadas, es decir, una solución se va construyendo por etapas, en base a decisiones adecuadas.
- ✚ Parte de una solución parcial, a partir de la cual se va construyendo la solución completa a partir de decisiones adecuadas.
- ✚ Aplica en aquellos problemas donde si existe un conjunto de soluciones y el orden en encontrarlas, no importa.

Programación Avanzada

- ✚ Tiene su origen en los años 50's por el matemático *Derrick Henry Lehmer* de la Universidad de California, *Berkeley* y formalizado en los años 60's por *R. J. Walker*, *Solomon Wolf Golomb* y *Leonard D. Baumert*

Derrick Henry Lehmer



Born February 23, 1905
Berkeley, California

Died May 22, 1991 (aged 86)
Berkeley, California

Nationality United States

Solomon W. Golomb



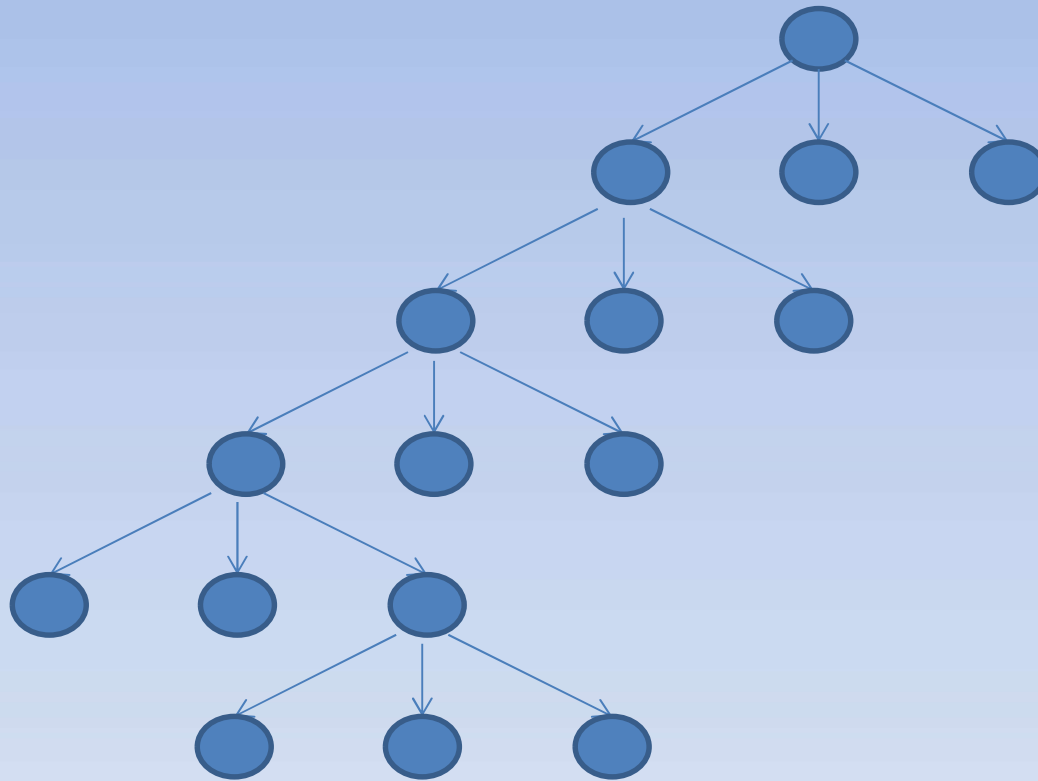
2014 studio portrait

Born Solomon Wolf Golomb
May 30, 1932
Baltimore, Maryland, U.S.

Died May 1, 2016 (aged 83)
Los Angeles, California, U.S.

Nationality American

Programación Avanzada



Árbol de búsqueda del algoritmo de backtracking

Programación Avanzada

Características de los problemas donde se puede aplicar este algoritmo.

- ✚ Existe una forma de saber si una solución encontrada es o no correcta.
- ✚ Existe una forma de generar soluciones parciales.
- ✚ El problema ofrece un conjunto de decisiones para generar las posibles soluciones parciales.

Programación Avanzada

Algoritmo.

- ✚ Elegir una solución parcial del conjunto inicial de opciones de solución (triviales).
- ✚ Acompletar la solución parcial anterior en base al conjunto de decisiones o reglas que ofrece el problema, para construir la solución final u otra solución parcial. Esto da origen a otra solución parcial o a la solución final. El estado final es la terminación del problema o solución completa.
Si esta es una solución, terminar.
Si no, aplicar de nuevo el primer paso.

Programación Avanzada

Algoritmo.

- ✚ Si no se puede completar la solución parcial encontrada, porque ya no hay más posibilidades en base a las decisiones o reglas que ofrece el problema para construir la solución, entonces considerar otra de las alternativas de solución parcial encontradas en el primer paso. Ir al segundo paso.
- ✚ Ver algoritmo en el documento:

Algoritmo BackTracking y Divide-Venceras.pdf